REAL-TIME EMOTION DETECTION AND MOOD TRACKING APPLICATION

UI design- Flutter(CS553PC) PROJECT REPORT

Submitted in partial fulfillment of the requirements for the

BACHELOR OF TECHNOLOGY

in

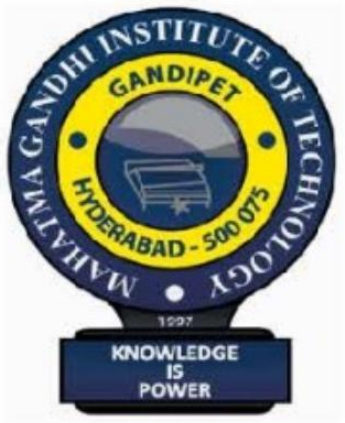
COMPUTER SCIENCE AND ENGINEERING(AIML)

by

KANAMUKKALA SUHAIL - 24261A6625

Under the guidance of

Mrs. K. Divija
(Assistant Professor)



Department of Emerging Technologies

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, A.P .)

Chaitanya Bharathi P.O. Gandipet, Hyderabad – 500 075

2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (A)

(Affiliated to Jawaharlal Nehru Technological University, Hyderabad, A.P .)

Chaitanya Bharathi P.O. Gandipet, Hyderabad – 500 075

DEPARTMENT OF EMERGING TECHNOLOGIES

CERTIFICATE

This is to certify that the technical project report entitled "**REAL-TIME EMOTION DETECTION AND MOOD TRACKING APPLICATION**" is a Bonafide academic work prepared by KANAMUKKALA SUHAIL- 24261A6625

In partial fulfilment of the requirements for the Bachelor of Technology programme at Mahatma Gandhi Institute of Technology, Hyderabad, during the academic year 2026.

The report is submitted in partial fulfilment of the requirements for the award of Bachelor of Technology by Jawaharlal Nehru Technological University, Hyderabad during the academic year 2025-2026. The work presented has not been submitted elsewhere for the award of any degree or diploma.

Mrs. K. DIVIJA
Assistant Professor

Dr. M. RAMA BAI
Head of the Department

Abstract

It is a Flutter-based real-time emotion detection and mood tracking interface developed using the Dart programming language. The application allows a user to express their current feelings by entering free-form text into a text field. As the text changes, the application analyzes it and identifies emotion-related keywords from a predefined local emotion database.

The current implementation contains twelve emotion categories: sad, angry, stress, happy, anxious, lonely, grateful, excited, hopeful, confused, tired, and proud. Each category is configured with a display label, emoji, color, gradient, and a list of keywords. The application converts the entered text to lowercase and uses regular-expression word-boundary matching to count occurrences of configured keywords. When one or more keywords match, the corresponding emotion tags are displayed with their matching counts.

MoodSpace provides a responsive scrolling interface with light and dark themes, a text-entry area, an empty-state message, a neutral-state card, and visually styled emotion tags. The application uses local in-memory data and does not require a backend, external API, database, or trained machine learning model for its current emotion detection workflow.

The project demonstrates a basic rule-based approach to text-based emotion identification. Its results depend on the predefined vocabulary and exact matching rules. It does not understand sentence meaning, context, sarcasm, negation, or the intensity of an emotion beyond counting keyword matches. Therefore, the displayed tags are indicative matches and should not be treated as a psychological assessment or definitive interpretation of a person's feelings.

Table of Contents

Section	Page
1. Introduction	5
2. Objectives	6
3. System Overview	7
4. Technology Stack	9
5. Data and Implementation Notes	10
6. Code Explanation	11
7. Results and Testing	18
8. Limitations and Future Enhancements	20
9. Conclusion	21
References	22

1. Introduction

1.1 Background

People often describe their feelings through short messages, personal notes, or longer paragraphs. A digital interface that recognizes emotion-related words can provide a simple way to reflect the language used by a person. Flutter is a UI framework that supports the development of applications from a shared Dart codebase, while Dart provides the programming language used to implement the interface and its local logic.

MoodSpace applies a predefined keyword-based method to text entered by the user. The application continuously checks the text for words associated with configured emotion categories and displays the categories whose keywords are found. The interface uses emojis, colors, gradients, and tag counts to make the detected matches easy to notice.

1.2 Overview of the Proposed System

MoodSpace is a single-screen Flutter application. Its primary workflow is based on text input and immediate visual feedback.

1. Text Input: Accepts a free-form description of how the user is feeling.
2. Emotion Configuration: Stores twelve emotion categories and their associated keywords and visual properties.
3. Text Analysis: Converts the input to lowercase and checks each configured keyword using regular expressions with word boundaries.
4. Emotion Tag Generation: Creates an EmotionData object for each category with at least one keyword match.
5. Result Presentation: Displays matching tags, emojis, and occurrence counts.
6. Empty and Neutral States: Shows guidance before text is entered and a neutral card when text is entered but no configured keyword matches.
7. Theme Support: Provides light and dark themes using Flutter Material 3 and system theme selection.

1.3 Problem Statement

Users may want a simple interface that reflects emotion-related language in text without manually selecting a mood category. A basic application can demonstrate how a predefined vocabulary and text-matching rules can convert written input into visual tags. The challenge is to perform matching while the user types, avoid partial-word false positives, support more than one detected category, and present results in a readable interface.

The project addresses these requirements through a local keyword-matching workflow. It does not attempt to solve the broader problem of understanding human emotion from natural language with contextual or clinical accuracy.

1.4 Proposed Solution

The proposed solution is MoodSpace, a Flutter application that monitors changes to a text field through its onChanged callback. The callback invokes _analyzeInput, which scans the text against the keyword list for each configured emotion. A regular expression using word boundaries is created for each keyword, and all matches are counted. The state is then updated so the interface displays the current set of matching emotion tags.

2. Objectives

The main objective of MoodSpace is to develop a simple, interactive Flutter application that identifies predefined emotion-related keywords in user-entered text and presents the matching categories immediately.

2.1 Real-Time Text Entry

To provide a text field where users can enter a description of their feelings. The application uses the TextEditingController to manage the entered text and the onChanged callback to trigger analysis whenever the text changes.

2.2 Predefined Emotion Categories

To maintain a local collection of emotion configurations for twelve categories: sad, angry, stress, happy, anxious, lonely, grateful, excited, hopeful, confused, tired, and proud.

2.3 Keyword-Based Analysis

To compare the entered text against the keywords configured for each emotion category. The current implementation uses lowercase conversion and regular-expression word-boundary matching.

2.4 Multiple Emotion Detection

To allow more than one emotion category to be detected from the same text when keywords from multiple categories are present.

2.5 Match Count Display

To count the number of occurrences of matching keywords for each category and display an x-count indicator when the intensity value is greater than one.

2.6 Clear and Understandable Interface

To present emotion results using category labels, emojis, colors, gradient backgrounds, and a responsive Wrap layout.

2.7 Light and Dark Theme Support

To support system-based theme selection through ThemeMode.system, with separate light and dark ThemeData configurations.

2.8 Local Execution

To perform the current analysis locally in Dart without requiring a remote API, backend service, database, or network connection.

3. System Overview

The MoodSpace application is organized into logical parts within the supplied Dart source file. The main widget initializes the application, the stateful screen manages input and detected results, the emotion database defines matching rules and visual settings, and helper methods build the interface states.

3.1 Application Initialization Module

MoodSpaceApp is a StatelessWidget that creates the MaterialApp. It sets the application title to MoodSpace, disables the debug banner, uses ThemeMode.system, enables Material Design 3, selects an indigo color scheme seed, and defines light and dark themes. MoodTrackerScreen is configured as the home screen.

- Initializes the Flutter application using runApp(const MoodSpaceApp()).
- Sets the application title and theme behavior.
- Loads MoodTrackerScreen as the main interface.

3.2 Mood Tracker Screen Module

MoodTrackerScreen is a StatefulWidget because the displayed interface changes according to the user's text. Its state class, _MoodTrackerScreenState, stores the TextEditingController and the list of currently detected EmotionData objects.

- Maintains the text input controller.
- Maintains the current list of detected emotions.
- Owns the local emotion configuration map.
- Rebuilds the interface when detected emotion state changes.

3.3 Emotion Database Module

The _emotionDatabase map stores the configuration for each supported emotion. Each map entry uses a label such as #sad or #angry and contains an EmotionConfig object.

Emotion tag	Emoji	Example configured keywords
#sad	🌧️	sad, down, depressed, cry, lonely, unhappy, heartbroken, gloomy
#angry	🔥	angry, mad, furious, annoyed, hate, frustrated, irritated, rage
#stress	⚡	stressed, overwhelmed, anxious, worried, pressure, busy, panic, tired
#happy	✨	happy, joy, excited, great, awesome, content, good, blessed, calm
#anxious	😰	anxious, nervous, uneasy, restless, jittery, dread, apprehensive, tense, overthinking
#lonely	🥀	lonely, alone, isolated, abandoned, empty, disconnected, unloved, left out, miss
#grateful	🙏	grateful, thankful, appreciative, blessed, fortunate, touched, moved, humbled
#excited	🎉	excited, thrilled, eager, pumped, hyped, enthusiastic, cant wait, looking forward, stoked

#hopeful	🌱	hopeful, optimistic, positive, looking up, better, improving, faith, believe, trust
#confused	😳	confused, lost, unsure, uncertain, puzzled, stuck, doubt, questioning, torn
#tired	😴	tired, exhausted, drained, sleepy, fatigued, burnt out, burnout, worn out, no energy
#proud	🏆	proud, accomplished, achieved, successful, victorious, confident, capable, strong

The table shows representative keywords from the code. The complete lists are retained in the source-code appendix.

3.4 Text Analysis Module

The _analyzeInput(String text) method performs the emotion matching. It converts the input to lowercase, creates an empty detected list, and iterates through every entry in _emotionDatabase. For every configured keyword, it creates a regular expression using a word boundary before and after the escaped keyword. The method counts all matches and adds an EmotionData object when the count is greater than zero.

Word boundaries help prevent a short keyword from matching as part of a longer word. For example, the keyword 'sad' should not match inside 'saddle'. The method uses RegExp.escape so characters in configured keywords are treated as literal text.

3.5 Emotion Result Module

The _buildTagsList method uses Wrap to display all detected emotions. Each tag uses a LinearGradient based on the corresponding EmotionConfig, a rounded border, a shadow, and the configured emoji and label. If the intensity is greater than one, an additional x-count badge is shown.

3.6 Empty State and Neutral State Module

When the text field is empty after trimming whitespace, _buildEmptyState displays a message asking the user to start typing. When text is present but no configured keyword is detected, _buildNeutralCard displays the #neutral label. The neutral card is a display state; it is not an entry in the emotion database and is not added to _activeEmotions.

3.7 Theme and Layout Module

The build method checks the active theme brightness and adjusts text and container colors accordingly. The main screen uses Stack, SafeArea, SingleChildScrollView, Column, Row, ClipRRect, BackdropFilter, Container, AnimatedSwitcher, and Wrap to construct the interface.

4. Technology Stack

4.1 Flutter

Flutter is used to build the MoodSpace user interface. The supplied code uses MaterialApp, Scaffold, TextField, Container, Card-like visual structures, layout widgets, icons, and theme configuration.

- Builds the application interface.
- Manages widget composition and stateful UI updates.
- Provides Material Design 3 components and theme support.

4.2 Dart

Dart is the programming language used for the application logic. It defines the widgets, emotion configuration classes, matching method, and state variables.

- Stores emotion categories and keywords.
- Converts input text to lowercase.
- Creates and executes regular expressions.
- Counts keyword matches and updates state.

4.3 Material Design 3

The application enables useMaterial3 in both light and dark ThemeData configurations. The Material design widgets and styling options help provide a consistent interface.

4.4 Regular Expressions

The Dart RegExp class is used to find configured keywords in the entered text. The pattern is constructed as r'\b' + RegExp.escape(keyword) + r'\b'. This applies word-boundary matching and avoids many partial-word matches.

4.5 Flutter UI and Visual Effects

The code imports dart:ui to use ImageFilter for the blurred background behind the input panel. Flutter widgets such as ClipRRect and BackdropFilter create the glass-like input panel. LinearGradient and BoxShadow style the emotion tags.

Technology / API	Role in MoodSpace
Flutter	Application UI and widget framework
Dart	Application logic and data structures
Material Design 3	UI components and theme configuration
TextEditingController	Reads and clears text input
RegExp	Matches configured emotion keywords
StatefulWidget / setState	Updates the displayed emotion results
ImageFilter / BackdropFilter	Creates the blurred glass-like input panel
LinearGradient / BoxShadow	Styles the emotion tags

5. Data and Implementation Notes

5.1 Data Management

The application stores its emotion definitions in the _emotionDatabase map. The map is initialized in the state class and contains EmotionConfig objects. The current detected results are stored in the _activeEmotions list. Both are local in-memory data structures.

5.2 Input Data

The primary input is a string entered by the user in the TextField. The TextEditingController named _controller is attached to the field. The text field supports up to five visible lines through maxLines: 5 and triggers _analyzeInput through onChanged.

5.3 Emotion Configuration Data

Every EmotionConfig contains a String label, String emoji, Color color, List<Color> gradient, and List<String> keywords. The color and gradient are used for visual presentation, while the keywords are used by the analysis method.

5.4 Keyword Matching Process

8. The text is received by _analyzeInput.
9. The text is converted to lowercase using toLowerCase().
10. A new empty List<EmotionData> is created.
11. The method iterates over each emotion configuration.
12. For each keyword, a regular expression is created with word boundaries and RegExp.escape.
13. All occurrences of that keyword are counted using allMatches().length.
14. If the total match count for an emotion is greater than zero, an EmotionData object is added to the detected list.
15. setState updates _activeEmotions, causing the UI to rebuild with the new result.

5.5 Match Count and Display

The intensity field in EmotionData stores the total number of keyword matches counted for one emotion category. It is not a medically validated measure of emotional intensity. The UI displays xN only when intensity is greater than one; a single match is shown without a count badge.

5.6 Data Persistence

The supplied code does not implement permanent data storage. It does not use SQLite, Firebase, shared preferences, a REST API, or another database. The application does not save mood history between launches. The current detected tags exist only in the running widget state.

5.7 Processing Limitations

The matching process is based on keyword presence rather than sentence meaning. It can miss feelings expressed with words that are not in the configured lists. It may also detect a keyword in a sentence where the user is denying or quoting that feeling. Some keywords overlap across categories, so one input can produce more than one tag. The code does not implement sentiment scoring, a trained NLP model, or contextual disambiguation.

6. Code Explanation

6.1 main() Function

The main function calls runApp(const MoodSpaceApp()). This starts the Flutter widget tree and loads the root application widget.

6.2 MoodSpaceApp Class

MoodSpaceApp extends StatelessWidget because its own configuration does not change during normal use. Its build method returns MaterialApp with the title MoodSpace, system theme mode, Material 3 enabled, indigo color scheme seed, and MoodTrackerScreen as the home page.

6.3 MoodTrackerScreen and State

MoodTrackerScreen extends StatefulWidget. The corresponding _MoodTrackerScreenState class owns the controller, active emotion list, and emotion configuration map. It overrides dispose() to dispose of the TextEditingController before calling super.dispose(), which releases the controller's resources.

6.4 _analyzeInput() Method

This method is the central processing function. It scans each keyword list, builds a word-boundary regular expression, counts all matches, and creates EmotionData records for categories with at least one match. It then calls setState to update the displayed results.

6.5 build() Method

The build method determines whether the current theme is dark and returns a Scaffold. A Stack places a decorative purple orb behind the main content. SafeArea and SingleChildScrollView make the content fit within the visible screen. The header shows the MoodSpace title and a psychology icon.

6.6 Input Panel

The text entry panel uses ClipRRect and BackdropFilter with ImageFilter.blur. Its TextField is connected to _controller and calls _analyzeInput whenever the text changes. A clear icon is shown only when the controller text is not empty; pressing it clears the controller and calls _analyzeInput with an empty string.

6.7 Result Rendering

AnimatedSwitcher changes between the empty state, neutral state, and detected tags. _buildTagsList creates a Wrap of styled tags. The category's configured gradient, emoji, label, and match count are read from EmotionData and EmotionConfig.

6.8 EmotionConfig and EmotionData Classes

EmotionConfig holds the static information that defines an emotion category. EmotionData holds the configuration reference and the number of keyword matches found for the current input. This keeps the category metadata separate from the current match count.

6.9 Complete Source Code

The following appendix reproduces the supplied project.txt source code. It is included for traceability; the explanation above describes the behavior implemented in that source.

Appendix A – Complete Flutter Source Code

Source file: main.dart | Language: Dart

```dart
import 'dart:ui' show ImageFilter;
import 'package:flutter/material.dart';

void main() {
  runApp(const MoodSpaceApp());
}

class MoodSpaceApp extends StatelessWidget {
  const MoodSpaceApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'MoodSpace',
      debugShowCheckedModeBanner: false,
      themeMode: ThemeMode.system,
      theme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.light,
        colorSchemeSeed: Colors.indigo,
        fontFamily: 'Roboto',
      ),
      darkTheme: ThemeData(
        useMaterial3: true,
        brightness: Brightness.dark,
        colorSchemeSeed: Colors.indigo,
        scaffoldBackgroundColor: const Color(0xFF0F0F1A),
      ),
      home: const MoodTrackerScreen(),
    );
  }
}

class MoodTrackerScreen extends StatefulWidget {
  const MoodTrackerScreen({super.key});

  @override
  State<MoodTrackerScreen> createState() => _MoodTrackerScreenState();
}

class _MoodTrackerScreenState extends State<MoodTrackerScreen> {
  final TextEditingController _controller = TextEditingController();
  List<EmotionData> _activeEmotions = [];

  final Map<String, EmotionConfig> _emotionDatabase = {
    '#sad': EmotionConfig(
      label: '#sad',
      emoji: '🌧️',
      color: const Color(0xFF4A90E2),
      gradient: const [Color(0xFF4A90E2), Color(0xFF50E3C2)],
      keywords: [
        'sad', 'down', 'depressed', 'cry', 'lonely',
        'unhappy', 'heartbroken', 'gloomy',
      ],
    ),
    '#angry': EmotionConfig(
      label: '#angry',
      emoji: '🔥',
      color: const Color(0xFFFF4B4B),
      gradient: const [Color(0xFFFF4B4B), Color(0xFFFF8533)],
      keywords: [
        'angry', 'mad', 'furious', 'annoyed', 'hate',
        'frustrated', 'irritated', 'rage',
      ],
    ),
    '#stress': EmotionConfig(
      label: '#stress',
      emoji: '⚡',
      color: const Color(0xFFFFB300),
      gradient: const [Color(0xFFFFB300), Color(0xFFF76B1C)],
      keywords: [
        'stressed', 'overwhelmed', 'anxious', 'worried',
        'pressure', 'busy', 'panic', 'tired',
      ],
    ),
    '#happy': EmotionConfig(
      label: '#happy',
      emoji: '✨',
      color: const Color(0xFF00E676),
      gradient: const [Color(0xFF00C853), Color(0xFFB2FF59)],
      keywords: [
        'happy', 'joy', 'excited', 'great', 'awesome',
        'content', 'good', 'blessed', 'calm',
```

```
      ],
    ),
    '#anxious': EmotionConfig(
      label: '#anxious',
      emoji: '😰',
      color: const Color(0xFF7C4DFF),
      gradient: const [Color(0xFF7C4DFF), Color(0xFFB388FF)],
      keywords: [
        'anxious', 'nervous', 'uneasy', 'restless', 'jittery',
        'dread', 'apprehensive', 'tense', 'overthinking',
      ],
    ),
    '#lonely': EmotionConfig(
      label: '#lonely',
      emoji: '🫂',
      color: const Color(0xFF5C6BC0),
      gradient: const [Color(0xFF5C6BC0), Color(0xFF9FA8DA)],
      keywords: [
        'lonely', 'alone', 'isolated', 'abandoned', 'empty',
        'disconnected', 'unloved', 'left out', 'miss',
      ],
    ),
    '#grateful': EmotionConfig(
      label: '#grateful',
      emoji: '🙏',
      color: const Color(0xFF26A69A),
      gradient: const [Color(0xFF26A69A), Color(0xFF80CBC4)],
      keywords: [
        'grateful', 'thankful', 'appreciative', 'blessed',
        'fortunate', 'touched', 'moved', 'humbled',
      ],
    ),
    '#excited': EmotionConfig(
      label: '#excited',
      emoji: '🎉',
      color: const Color(0xFFFF6D00),
      gradient: const [Color(0xFFFF6D00), Color(0xFFFFD54F)],
      keywords: [
        'excited', 'thrilled', 'eager', 'pumped', 'hyped',
        'enthusiastic', 'cant wait', 'looking forward', 'stoked',
      ],
    ),
    '#hopeful': EmotionConfig(
      label: '#hopeful',
      emoji: '🌱',
      color: const Color(0xFF00BFA5),
      gradient: const [Color(0xFF00BFA5), Color(0xFFA7FFEB)],
      keywords: [
        'hopeful', 'optimistic', 'positive', 'looking up',
        'better', 'improving', 'faith', 'believe', 'trust',
      ],
    ),
    '#confused': EmotionConfig(
      label: '#confused',
      emoji: '🤔',
      color: const Color(0xFF8D6E63),
      gradient: const [Color(0xFF8D6E63), Color(0xFFD7CCC8)],
      keywords: [
        'confused', 'lost', 'unsure', 'uncertain', 'puzzled',
        'stuck', 'doubt', 'questioning', 'torn',
      ],
    ),
    '#tired': EmotionConfig(
      label: '#tired',
      emoji: '🥱',
      color: const Color(0xFF546E7A),
      gradient: const [Color(0xFF546E7A), Color(0xFFB0BEC5)],
      keywords: [
        'tired', 'exhausted', 'drained', 'sleepy', 'fatigued',
        'burnt out', 'burnout', 'worn out', 'no energy',
      ],
    ),
    '#proud': EmotionConfig(
      label: '#proud',
      emoji: '🏆',
      color: const Color(0xFFD4AF37),
      gradient: const [Color(0xFFD4AF37), Color(0xFFFFE082)],
      keywords: [
        'proud', 'accomplished', 'achieved', 'successful',
        'victorious', 'confident', 'capable', 'strong',
      ],
    ),
  };

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
```

```
  }

  void _analyzeInput(String text) {
    final lowerText = text.toLowerCase();
    final List<EmotionData> detected = [];

    _emotionDatabase.forEach((key, config) {
      int matches = 0;
      for (final keyword in config.keywords) {
        // Word-boundary match to avoid false positives
        // (e.g. "sad" inside "saddle").
        final pattern = RegExp(
          r'\b' + RegExp.escape(keyword) + r'\b',
          caseSensitive: false,
        );
        matches += pattern.allMatches(lowerText).length;
      }
      if (matches > 0) {
        detected.add(EmotionData(config: config, intensity: matches));
      }
    });

    setState(() {
      _activeEmotions = detected;
    });
  }

  @override
  Widget build(BuildContext context) {
    final isDark = Theme.of(context).brightness == Brightness.dark;

    return Scaffold(
      body: Stack(
        children: [
          // Background Gradient Orb (purple, top-right only)
          Positioned(
            top: -50,
            right: -50,
            child: Container(
              width: 200,
              height: 200,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                color: Colors.purple.withValues(alpha: 0.25),
              ),
            ),
          ),

          SafeArea(
            child: SingleChildScrollView(
              padding: const EdgeInsets.symmetric(
                horizontal: 24.0,
                vertical: 16.0,
              ),
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  const SizedBox(height: 12),
                  // Header
                  Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      Column(
                        crossAxisAlignment: CrossAxisAlignment.start,
                        children: [
                          Text(
                            'MoodSpace',
                            style: TextStyle(
                              fontSize: 28,
                              fontWeight: FontWeight.bold,
                              letterSpacing: -0.5,
                              color: isDark ? Colors.white : Colors.black87,
                            ),
                          ),
                          Text(
                            'Express yourself freely',
                            style: TextStyle(
                              fontSize: 14,
                              color: isDark ? Colors.white54 : Colors.black54,
                            ),
                          ),
                        ],
                      ),
                      CircleAvatar(
                        radius: 22,
                        backgroundColor: isDark
                            ? Colors.white10
                            : Colors.black.withValues(alpha: 0.05),
                        child: const Icon(
```

```
                        Icons.psychology_outlined,
                        size: 24,
                      ),
                    ),
                  ],
                ),

                const SizedBox(height: 32),

                // Main Input Box (Glassmorphism Effect)
                ClipRRect(
                  borderRadius: BorderRadius.circular(24),
                  child: BackdropFilter(
                    filter: ImageFilter.blur(sigmaX: 10, sigmaY: 10),
                    child: Container(
                      padding: const EdgeInsets.all(20),
                      decoration: BoxDecoration(
                        color: isDark
                            ? Colors.white.withValues(alpha: 0.05)
                            : Colors.white.withValues(alpha: 0.7),
                        borderRadius: BorderRadius.circular(24),
                        border: Border.all(
                          color: isDark
                              ? Colors.white.withValues(alpha: 0.1)
                              : Colors.black.withValues(alpha: 0.05),
                        ),
                        boxShadow: [
                          BoxShadow(
                            color: Colors.black.withValues(alpha: 0.04),
                            blurRadius: 20,
                            offset: const Offset(0, 10),
                          ),
                        ],
                      ),
                      child: Column(
                        crossAxisAlignment: CrossAxisAlignment.end,
                        children: [
                          TextField(
                            controller: _controller,
                            maxLines: 5,
                            style: const TextStyle(
                              fontSize: 16,
                              height: 1.5,
                            ),
                            decoration: InputDecoration(
                              hintText:
                              'How are you feeling right now? Write anything...',
                              hintStyle: TextStyle(
                                color: isDark
                                    ? Colors.white38
                                    : Colors.black38,
                              ),
                              border: InputBorder.none,
                            ),
                            onChanged: _analyzeInput,
                          ),
                          if (_controller.text.isNotEmpty)
                            IconButton(
                              icon: const Icon(
                                Icons.clear_rounded,
                                size: 20,
                              ),
                              onPressed: () {
                                _controller.clear();
                                _analyzeInput('');
                              },
                            ),
                        ],
                      ),
                    ),
                  ),
                ),

                const SizedBox(height: 32),

                // Results Header
                Row(
                  children: [
                    const Icon(
                      Icons.auto_awesome,
                      size: 18,
                      color: Colors.indigoAccent,
                    ),
                    const SizedBox(width: 8),
                    Text(
                      'DETECTED EMOTIONS',
                      style: TextStyle(
                        fontSize: 12,
                        fontWeight: FontWeight.bold,
```

```
                        letterSpacing: 1.2,
                        color: isDark ? Colors.white54 : Colors.black45,
                      ),
                    ),
                  ],
                ),

                const SizedBox(height: 16),

                // Dynamic Tags Grid / Empty State
                AnimatedSwitcher(
                  duration: const Duration(milliseconds: 300),
                  child: _controller.text.trim().isEmpty
                      ? _buildEmptyState(isDark)
                      : _activeEmotions.isEmpty
                      ? _buildNeutralCard(isDark)
                      : _buildTagsList(),
                ),
              ],
            ),
          ),
        ),
      ],
    ),
  );
}

Widget _buildEmptyState(bool isDark) {
  return Container(
    width: double.infinity,
    padding: const EdgeInsets.all(28),
    decoration: BoxDecoration(
      color: isDark
          ? Colors.white.withValues(alpha: 0.02)
          : Colors.black.withValues(alpha: 0.02),
      borderRadius: BorderRadius.circular(20),
      border: Border.all(
        color: isDark
            ? Colors.white10
            : Colors.black.withValues(alpha: 0.05),
        style: BorderStyle.solid,
      ),
    ),
    child: Center(
      child: Text(
        'Start typing above to auto-detect emotion tags in real-time.',
        textAlign: TextAlign.center,
        style: TextStyle(color: isDark ? Colors.white38 : Colors.black38),
      ),
    ),
  );
}

Widget _buildNeutralCard(bool isDark) {
  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 18, vertical: 12),
    decoration: BoxDecoration(
      color: isDark ? Colors.white10 : Colors.grey.shade200,
      borderRadius: BorderRadius.circular(30),
    ),
    child: const Row(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text('😐', style: TextStyle(fontSize: 18)),
        SizedBox(width: 8),
        Text(
          '#neutral',
          style: TextStyle(fontWeight: FontWeight.w600),
        ),
      ],
    ),
  );
}

Widget _buildTagsList() {
  return Wrap(
    spacing: 12,
    runSpacing: 12,
    children: _activeEmotions.map((data) {
      return Container(
        padding: const EdgeInsets.symmetric(
          horizontal: 18,
          vertical: 10,
        ),
        decoration: BoxDecoration(
          gradient: LinearGradient(
            colors: data.config.gradient,
            begin: Alignment.topLeft,
            end: Alignment.bottomRight,
```

```
            ),
            borderRadius: BorderRadius.circular(30),
            boxShadow: [
              BoxShadow(
                color: data.config.color.withValues(alpha: 0.35),
                blurRadius: 12,
                offset: const Offset(0, 6),
              ),
            ],
          ),
          child: Row(
            mainAxisSize: MainAxisSize.min,
            children: [
              Text(
                data.config.emoji,
                style: const TextStyle(fontSize: 16),
              ),
              const SizedBox(width: 8),
              Text(
                data.config.label,
                style: const TextStyle(
                  color: Colors.white,
                  fontWeight: FontWeight.bold,
                  fontSize: 14,
                ),
              ),
              if (data.intensity > 1) ...[
                const SizedBox(width: 6),
                Container(
                  padding: const EdgeInsets.symmetric(
                    horizontal: 6,
                    vertical: 2,
                  ),
                  decoration: BoxDecoration(
                    color: Colors.white.withValues(alpha: 0.3),
                    borderRadius: BorderRadius.circular(10),
                  ),
                  child: Text(
                    'x${data.intensity}',
                    style: const TextStyle(
                      color: Colors.white,
                      fontSize: 11,
                      fontWeight: FontWeight.bold,
                    ),
                  ),
                ),
              ],
            ],
          ),
        );
      }).toList(),
    );
  }
}

class EmotionConfig {
  final String label;
  final String emoji;
  final Color color;
  final List<Color> gradient;
  final List<String> keywords;

  EmotionConfig({
    required this.label,
    required this.emoji,
    required this.color,
    required this.gradient,
    required this.keywords,
  });
}

class EmotionData {
  final EmotionConfig config;
  final int intensity;

  EmotionData({required this.config, required this.intensity});
}
```

7. Results and Testing

7.1 Application Output

The supplied implementation produces three principal interface states based on the text field content:

- Empty input: Displays the instruction 'Start typing above to auto-detect emotion tags in real-time.'
- Non-empty input with no configured keyword match: Displays the #neutral card.
- Input with one or more keyword matches: Displays the matching emotion tags with their emoji, label, gradient styling, and optional x-count.

7.2 Functional Test Cases

Test case	Input / action	Expected behavior from code
TC-01	Launch application	MoodSpace screen is displayed with system-selected light or dark theme.
TC-02	Leave text field empty	Empty-state instruction is displayed.
TC-03	Enter a sentence containing 'sad'	The #sad tag is added to the detected emotion list.
TC-04	Enter text containing 'angry' and 'furious'	The #angry tag is detected and its count can display x2.
TC-05	Enter text with keywords from two categories	Both matching category tags can be displayed.
TC-06	Enter text with no configured keyword	The #neutral display card is shown.
TC-07	Enter 'saddle' only	The keyword 'sad' should not match as a substring because of word boundaries.
TC-08	Press the clear icon after entering text	The field is cleared and the empty state is restored.
TC-09	Switch system appearance	The application uses the corresponding light or dark theme.
TC-10	Enter whitespace only	After trim(), the empty-state branch is selected.

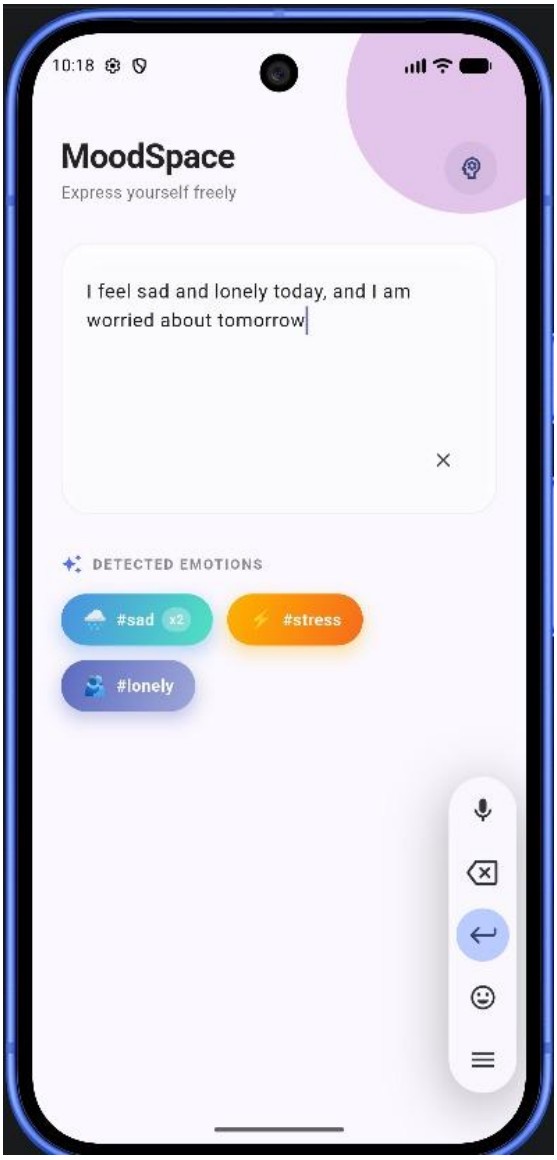
7.3 Example Demonstration

Example input: “I feel sad and lonely today, and I am worried about tomorrow.”

Expected keyword-based result: #sad, #lonely, and #stress or #anxious may be detected only if the exact configured keywords occur. In this example, 'sad', 'lonely', and 'worried' are configured; 'worried' belongs to both #stress and #stress? In the supplied code, 'worried' is configured under #stress, while #anxious contains 'anxious' but not 'worried'. Therefore the expected tags are #sad, #lonely, and #stress.

This example illustrates that the same keyword can only contribute to the category lists in which it is configured. The application reports matching tags rather than selecting one single final emotion.

7.4 Result Interpretation



8. Limitations and Future Enhancements

8.1 Current Limitations

- Keyword-only detection: The application identifies configured words but does not understand complete sentence meaning.
- No contextual analysis: Negation, sarcasm, quotations, and surrounding context are not interpreted.
- Fixed vocabulary: A feeling may not be detected if the user uses a word that is absent from the keyword list.
- Overlapping categories: A keyword may belong to more than one category, and multiple tags may be shown.
- No trained ML model: The current source does not load or call a machine learning or NLP model.
- No backend or database: The application does not save entries or mood history permanently.
- No solution recommendations: The current code detects and displays emotion tags only; it does not generate coping advice or personalized recommendations.
- No authentication: The application does not include login, registration, or user accounts.

8.2 Expanded Keyword Vocabulary

The emotion keyword lists can be expanded with additional words, common expressions, and carefully reviewed variations. Any expansion should be tested for overlap and false matches so that new keywords do not create misleading tags.

8.3 Context-Aware Natural Language Processing

A future version could use a natural language processing model to analyze sentence context, negation, and relationships between words. This would be a separate enhancement; it is not part of the current implementation.

8.4 Machine Learning Integration

A trained text-classification model could be introduced after collecting and validating a suitable labeled dataset. The model would require training, evaluation, and testing on representative text. The current keyword-matching implementation should be treated as a baseline rather than as a trained classifier.

8.5 Mood History and Database

A database such as SQLite or a cloud database could be added to save user-approved mood entries and timestamps. This would enable a history screen and trend visualization. Such storage is not currently present in the supplied code.

8.6 Personalized Suggestions

A future release could provide general wellbeing suggestions or links to supportive resources based on a detected category. Recommendations should be clearly presented as general information, not diagnosis or treatment. This feature is not implemented in the current source.

8.7 Privacy and User Control

If text entries are stored or sent to a service in a future version, the application should explain how the data is used, request appropriate consent, and provide deletion controls. The current implementation performs matching locally and does not send the entered text to a remote service.

9. Conclusion

MoodSpace demonstrates a Flutter-based interface for real-time, keyword-driven emotion tag detection. The application accepts free-form text and checks it against twelve predefined emotion categories. It uses lowercase conversion, regular-expression word-boundary matching, match counting, and StatefulWidget updates to display matching tags as the user types.

The project also demonstrates Flutter interface concepts including Material Design 3, light and dark themes, a TextEditingController, responsive scrolling layout, AnimatedSwitcher, BackdropFilter, gradients, and reusable data classes. The source code keeps the current workflow local and does not depend on a backend or database.

The current system is a rule-based demonstration rather than a trained machine learning emotion classifier. Its output depends on the configured vocabulary and should be interpreted as keyword matches, not as a definitive assessment of a user's emotional state. Future work may add contextual NLP, a validated ML model, persistent mood history, and optional wellbeing resources while maintaining clear privacy and accuracy limitations.

References

1. Flutter Documentation – https://docs.flutter.dev/
2. Dart Programming Language Documentation – https://dart.dev/
3. Flutter Material Design Documentation – https://docs.flutter.dev/ui/design/material
4. Flutter API Reference – https://api.flutter.dev/
5. Dart API Reference – https://api.dart.dev/
6. Dart RegExp API – https://api.dart.dev/dart-core/RegExp-class.html
7. Flutter TextField API – https://api.flutter.dev/flutter/material/TextField-class.html
8. Flutter StatefulWidget API – https://api.flutter.dev/flutter/widgets/StatefulWidget-class.html